

Visitor Design Pattern

Revision Sheet

1 1. Core Problem

Suppose we have a relatively stable hierarchy of objects:

$$\text{Element} \rightarrow \{\text{Circle, Rectangle, Triangle, } \dots\}$$

but many operations need to be performed on these objects:

$$\text{Draw, Export, Area, Print, Serialize, } \dots$$

If every operation is placed inside every element class, the classes become overloaded with unrelated responsibilities.

Core problem:

We have a set of object types and want to repeatedly add **new operations** over those objects without modifying all the object classes every time.

Typical example: Compiler AST

$$\text{AST Nodes} = \{\text{BinaryOp, Assignment, Identifier, Literal, } \dots\}$$

while operations may be:

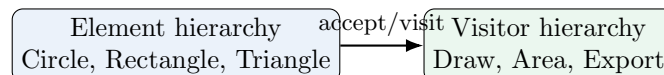
$$\text{Type Checking, TAC Generation, Optimization, Pretty Printing, } \dots$$

2 2. Central Idea

The Visitor pattern:

separates operations from the objects on which they operate.

Instead of putting every operation inside the element classes, represent each operation as a separate **Visitor**.



Visitor = stable element hierarchy + frequently changing operations

3 3. Important Participants

The GoF Visitor pattern has five conceptual participants:

1. **Visitor**
2. **ConcreteVisitor**
3. **Element**
4. **ConcreteElement**
5. **ObjectStructure**

Visitor

Defines one operation for every concrete element type:

```
class Visitor {
public:
    virtual void visit(Circle&) = 0;
    virtual void visit(Rectangle&) = 0;
    virtual void visit(Triangle&) = 0;

    virtual ~Visitor() = default;
};
```

ConcreteVisitor

Implements one complete operation:

```
class DrawVisitor : public Visitor {
public:
    void visit(Circle&) override;
    void visit(Rectangle&) override;
    void visit(Triangle&) override;
};
```

Examples of ConcreteVisitors:

DrawVisitor, AreaVisitor, ExportVisitor, TACVisitor,...

Element

Provides the entry point for a visitor:

```
class Element {
public:
    virtual void accept(Visitor&) = 0;
    virtual ~Element() = default;
};
```

ConcreteElement

Each concrete element implements `accept()`:

```
class Circle : public Element {
public:
    void accept(Visitor& v) override {
        v.visit(*this);
    }
};
```

Likewise:

```
class Rectangle : public Element {
public:
    void accept(Visitor& v) override {
        v.visit(*this);
    }
};
```

ObjectStructure

Contains/traverses the elements and lets a visitor operate on them.

It may be:

List, Tree, Composite, Document, AST,...

For example:

```
class Document {
    vector<Element*> elements;

public:
    void accept(Visitor& v) {
        for (Element* e : elements)
            e->accept(v);
    }
};
```

4. The Most Important Part: accept()

The purpose of `accept()` is to solve the problem that an external visitor initially sees only the base type.
Suppose:

```
Element* e = new Circle();
```

We cannot rely on:

```
visitor.visit(*e);
```

to select `visit(Circle&)` because overload resolution uses the **static type**, not the runtime type.
Instead:

```
e->accept(visitor);
```

Because `accept()` is virtual, runtime polymorphism selects:

`Circle::accept()`

Inside `Circle`:

```
v.visit(*this);
```

Here `this` is a `Circle*`, so `*this` is a `Circle&`.
Hence the call becomes:

```
v.visit(Circle&);
```

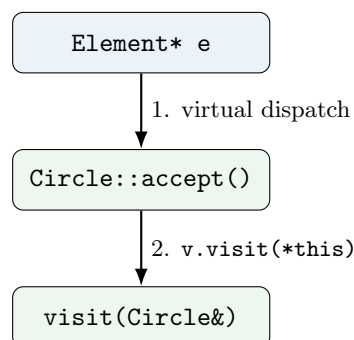
5. Double Dispatch

This is the heart of the Visitor pattern.

Visitor implements **double dispatch** using:

1. virtual dispatch on `accept()`
2. overload resolution on `visit(...)`

Complete flow



So:

```
Element* → Circle::accept() → visit(Circle&)
```

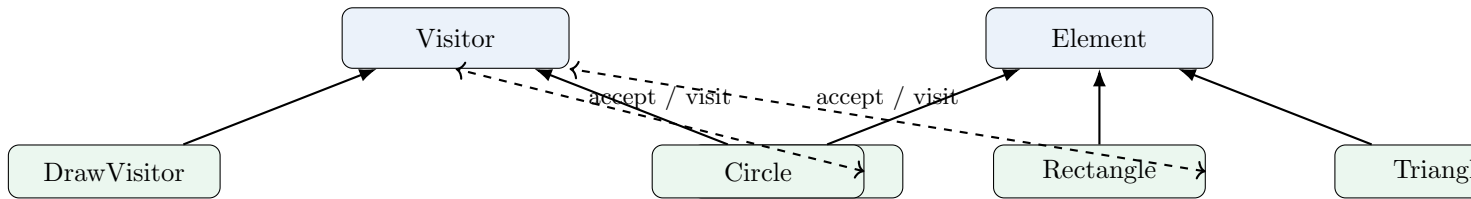
Why “double”?

1. First dispatch identifies the concrete **Element**.
2. Second dispatch selects the corresponding operation in the **Visitor**.

Element type + Visitor operation

C++ does not natively provide general double dispatch.
The Visitor pattern **constructs** it using virtual functions plus overloaded `visit()` functions.

6 6. Complete Structural Diagram



7 7. ObjectStructure

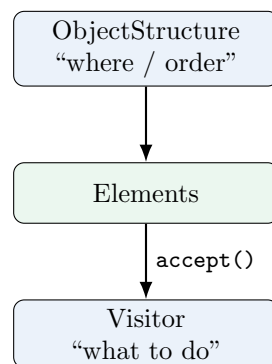
The ObjectStructure determines:

Which objects are visited and in what order

The Visitor determines:

What operation is performed

Hence:



For a tree such as a compiler AST:

AST → visit every node → perform operation according to node type

8 8. Why Visitor Fits Compiler ASTs

A compiler AST often has a relatively stable node hierarchy:

ASTNode → {BinaryOp, UnaryOp, Assignment,
Identifier, IntLiteral, FloatLiteral, ...}

But many passes may operate on the same AST:

Type Checking
Pretty Printing
TAC Generation
Optimization
Code Generation

Therefore:

Stable AST structure + many evolving passes $\Rightarrow$ Visitor is a natural fit

9 9. When Should We Use Visitor?

Use Visitor when:

1. The set of element types is relatively stable.
2. New operations over those elements are added frequently.
3. Operations need different behavior for different concrete elements.
4. You want to keep algorithms separate from the element classes.

Rule of Thumb

Stable objects + changing operations $\Rightarrow$ Visitor

If instead:

Changing objects + stable operations

Visitor is generally a poor fit.

10 10. Advantages

1. Easy to add new operations

A new operation is usually a new ConcreteVisitor.

2. Groups related operation logic

All logic for one operation can live in one visitor.

3. Separates algorithms from object structure

Elements represent data/structure; Visitors represent operations.

4. Can accumulate state

A visitor can maintain results/state while traversing an entire structure.

5. Works well on heterogeneous structures

A single visitor can process many different concrete element types.

11 11. Disadvantages

1. Adding a new Element is difficult

A new concrete Element requires a new `visit(NewElement&)` in the Visitor interface and usually in every ConcreteVisitor.

2. Can weaken encapsulation

Visitors may need access to internal state of elements.

3. Tight coupling to the Element hierarchy

Visitor must know the concrete Element types.

4. Visitor interface may become large

Large element hierarchies produce many `visit()` functions.

5. More complicated than ordinary polymorphism

`accept()` + `visit()` + double dispatch introduce additional design complexity.

12 12. The Fundamental Trade-off

This is the single most important consequence of Visitor:

Change	Visitor
Add a new operation	Easy
Modify an existing operation	Easy / localized
Add a new element type	Difficult
Modify the element hierarchy	Potentially affects all visitors

Visitor makes operations extensible at the cost of making
the element hierarchy less extensible.

13 13. Open/Closed Principle – Important Nuance

Visitor is particularly good for the **operation dimension**.

New operation → New Visitor → Existing Elements unchanged

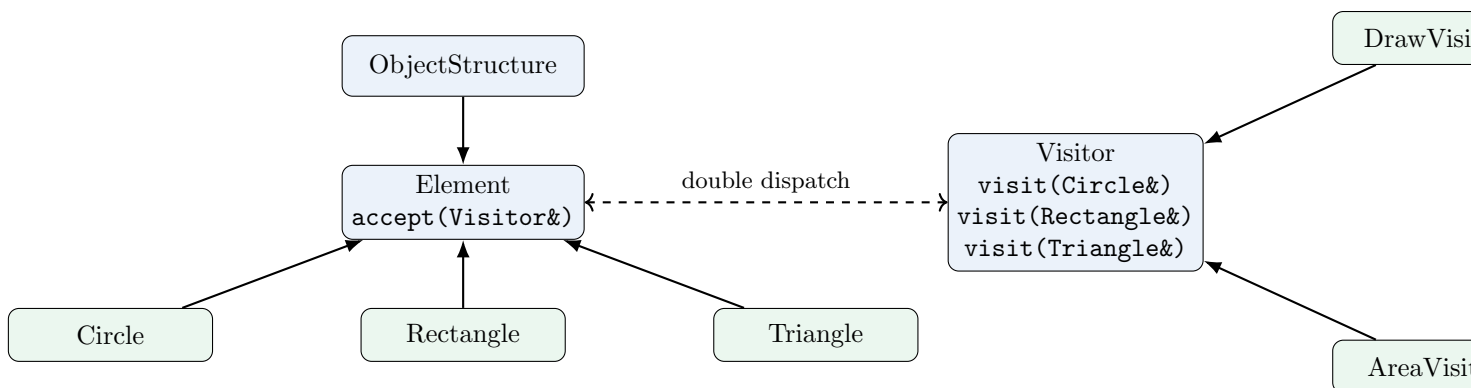
But for the element dimension:

New Element → Modify Visitor interface → Modify all ConcreteVisitors

Therefore Visitor is not “universally open for extension”.

It deliberately chooses which axis is easy to extend.

14 14. The Entire Pattern in One Picture



15 15. Revision in 6 Lines

1. Visitor **separates operations from object structure**.
2. Each Element provides `accept(Visitor&)`.
3. Each ConcreteElement calls `visitor.visit(*this)`.
4. This implements **double dispatch**.
5. It makes **adding operations easy**.
6. It makes **adding element types harder**.

16 16. Final Mental Model

Whenever you see:

Many operations + relatively stable class hierarchy

think:

VISITOR

And remember the core call chain:

`element->accept(visitor) → ConcreteElement::accept() → visitor.visit(*this) → visit(ConcreteElement&)`

Visitor = “Add operations without changing the elements”